

编译WASM版的ffmpeg

docker安装

Mac/Window/Linux 都可以, 以mac举例

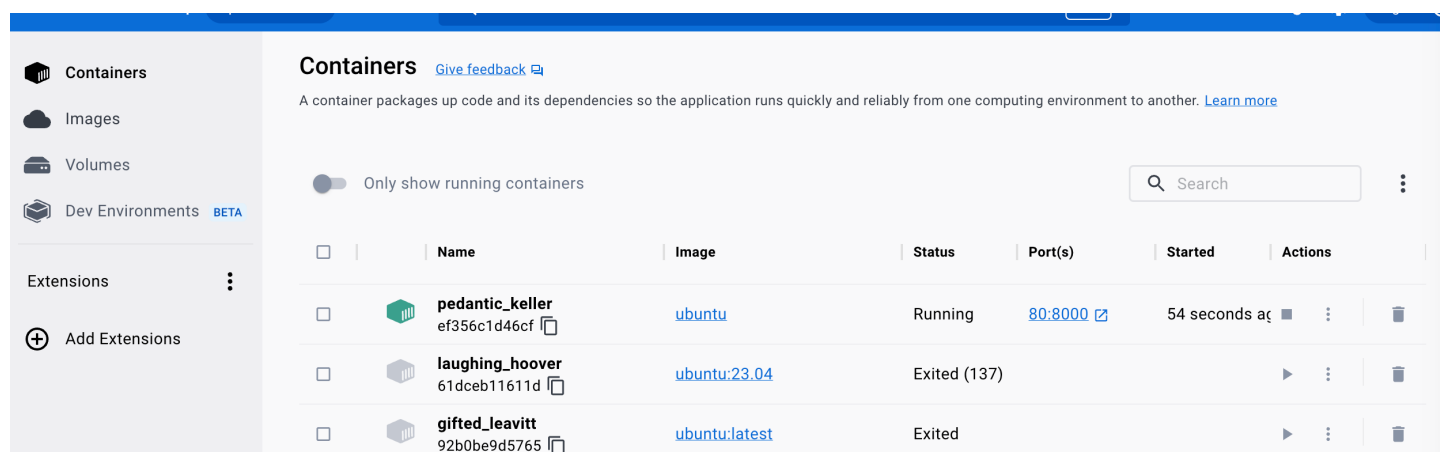
下载docker

<https://www.docker.com>

启动docker并安装最新的ubuntu, 如果docker里没有会自动安装

```
1 docker container run -it -p 80:8000 ubuntu
```

这种方式每次启动都会创新一个新的镜像, 第一次启动之后, 再次启动, 不要执行, 在docker的客户端点击启动, 然后再执行进入docker命令



接下来我们使用该命令进入一个已经在运行的容器, 点击Containers

```
1 docker ps
2 docker exec -it 775c7c9ee1e1 /bin/bash
```

进入docker

安装启动ubuntu

```
1 docker pull ubuntu:23.04
```

```
2 docker run -itd ubuntu:23.04 bash
3 61deeb11611d24a7c6cb9cdd00f6267d968cc1aaac1b5f6f46a01427d53f97a3
4 docker exec -it
5 61deeb11611d24a7c6cb9cdd00f6267d968cc1aaac1b5f6f46a01427d53f97a3 bash
```

环境安装

```
1 apt-get install vim xz-utils bzip2 iputils-ping net-tools nginx cmake
python3.10
```

如果上面git安装不成功, 使用下面命令

```
1 apt-get update
2 apt-get install git
```

安装emsdk

```
1 git clone https://github.com/emscripten-core/emsdk.git
2 cd emsdk/
3 ./emsdk install latest
4 ./emsdk activate latest
5 source ./emsdk_env.sh
```

cat /etc/os-release

查看docker名字

```
1 docker ps
2 CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS
   NAMES
3 ef356c1d46cf   ubuntu   "bash"    17 hours ago    Up 17 hours    0.0.0.0:80->8000/tcp
   pedantic_keller
```

从docker拷贝文件到本地

- 1 从容器cp到本地
- 2 `docker cp ef356c1d46cf:/home/emsdk/test /Users/jason/Desktop/docker`
- 3 从本地cp到容器
- 4 `docker cp /Users/jason/Desktop/docker/ffwasm ef356c1d46cf:/home/wasm/decoder`

emcc 编译源码到wasm

- 1 `emcc hello.c -o hello.html`

wasm debug

1. 安装插件

<https://chrome.google.com/webstore/detail/cc%2B%2B-devtools-support-dwa/pdcpmagijalfljmkmjngeonclgbbannb/related>

2. 编译带有调式信息

`emcc -g test.c -fdebug-compilation-dir='.' -o test.html`

3. 发布

本地启动/访问

`emrun --no_browser --port 8000 hello.html`

<http://localhost:8000/test.html>

docker启动/访问

`emrun --no_browser --port 8000 hello.html`

<http://localhost/test.html> //不需要加80

4. 打开Chrome调试

设置

偏好设置

工作区

实验

忽略列表

设备

节流

位置

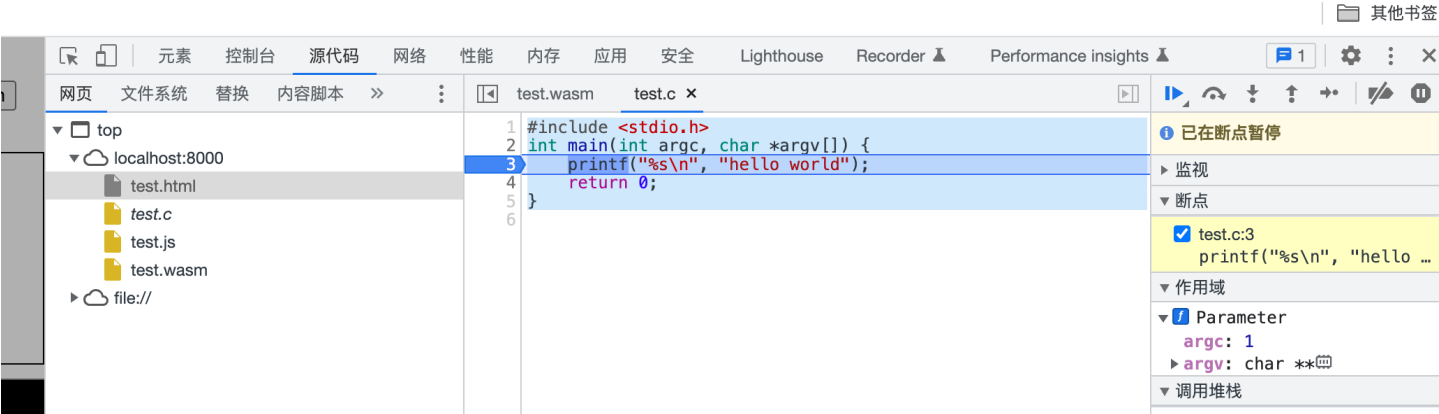
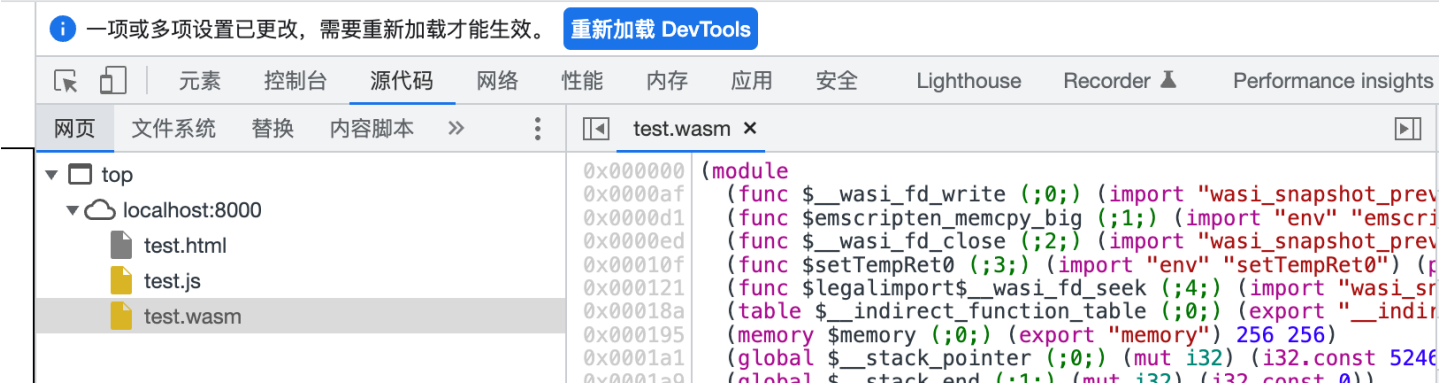
快捷键

实验

过滤

警告： 这些实验可能不稳定或不可靠，因此可能需要您重启 DevTools。

☒ WebAssembly Debugging: Enable DWARF support



ubuntu环境下编译ffmpeg

ubuntu系统可以是docker下的ubuntu或者物理机ubuntu

1. ubuntu系统下进行交叉编译wasm支持的静态库

git clone <https://git.ffmpeg.org/ffmpeg.git> ffmpeg

查看版本, 找到最新版本

git branch -r

git checkout origin/release/5.1

交叉编译

- 1 指定c语言编译工具 --cc="emcc"
- 2 指定c++语言编译工具 --cxx="em++"
- 3 指定创建静态库的工具 --ar="emar" //不指定编译工具, 默认会找主机的编译工具
- 4 输出目标 --enable-cross-compile
- 5 指定程序运行的操作系统 --target-os=none //none 所有操作系统都支持
- 6 cpu架构 --arch=x86_32
- 7 cpu型号 --cpu=generic //不指定型号, 都支持

裁剪ffmpeg

- 1 --disable-avformat
- 2 --disable-avdevice
- 3 --disable-avfilter
- 4 --disable-swresample
- 5 --disable-programs //ffmpeg ffplay ffprobe
- 6 --disable-debug
- 7 --disable-doc
- 8 --disable-everything
- 9
- 10 --enable-decoder=hevc --enable-parser=hevc
- 11 --enable-decoder=h264 --enable-parser=h264

chmod +x build_ffmpeg.sh

- 1 ...
- 2 ./libavutil/x86/timer.h:39:24: error: invalid output constraint '=a' in asm
- 3 : "=a" (a), "=d" (d));

禁用--disable-asm, 禁用--disable-x86asm好像不行

把程序拷贝到ubuntu, 在ubuntu上编译ffmpeg的wasm版本

```
1 clang -g -o ff_decode_video ff_decode_video.c
2 clang -g -o ff_decode_video ff_decode_video.c -lavutil -lavformat -lavcodec -
  lswscale
```

ffmpeg 封装wasm播放器开发, 目前已完成的功能, 支持视频播放, 暂停, seek

1. wasm debug 版本编译, 支持在Chrome里直接可以断点调试c/c++代码
2. 逐帧解码, 任意时间点取帧
3. seek解码, 任意时间点取帧
4. wasm与js端数据交互与渲染. 输出yuv420p数据, 渲染到WebGL Canvas上. 测试demo完成

花椒前端基于WebAssembly 的H.265播放器研发随着近些年直播技术的不断更新迭代, 高画质、低带宽、低成本成为直 - 掘金